목차

1 환경 변수 및 OpenAI 클라이언트 설정

2 DB 연결 함수

3 임베딩 함수

4 사용자 입력 포맷 변환 함수

5 코사인 유사도 계산

6 유사 공지 검색

7 실행부

8 서버 연결 (Flask)

9 마무리 정리

환경 변수 및 OpenAI 클라이언트 설정

API 키를 안전하게 관리하고 OpenAI 클라이언트 초기화하기

1. 환경 변수 설정 및 클라이언트 초기화

```
from dotenv import
load_dotenv import os
from openai import OpenAI

load_dotenv()


api_key =
os.getenv("OPENAI_API_KEY")


client = OpenAI(api_key=api_key)
```

환경 변수 및 OpenAI 클라이언트 설정

API 키를 안전하게 관리하고 OpenAI 클라이언트 초기화하기

2 .env 파일 작성 방법

```
OPENAI_API_KEY=sk-proj-5YaqrUVZ...
DB_HOST=uoscholar.cdkke4m4o6zb.ap-northeast-2.rds.amazonaws.com
DB_USER=admin
DB_PASSWORD=dongha1005!
DB_NAME=uoscholar_db
DB_PORT=3306
```

환경 변수 사용 이점

- 민감한 정보가 소스 코드에 직접 포함되지 않음
- 개발/테스트/프로덕션 환경별 설정 분리

주의사항

- 키=값 형식에서 등호 주변에 공백 사용 금지 잘
- 못된 예: OPENAI_API_KEY = key
- 올바른 예: OPENAI_API_KEY=key

DB 연결 함수

MySQL DB에 안전하게 연결하는 함수 구현

```
def get_connection():
    try:
        return mysql.connector.connect(
            host=os.getenv("DB_HOST"),
            user=os.getenv("DB_USER"),
            password=os.getenv("DB_PASSWORD"),
            database=os.getenv("DB_NAME"),
            port=int(os.getenv("DB_PORT"))
        )
    except mysql.connector.Error as e:
        print(f"DB 연결 실패: {e}")
        return None
```

함수 동작 방식

- 환경 변수에서 DB 연결 정보를 가져옴
- os.getenv()로 .env 파일의 변수 접근
- 연결 객체 생성 후 반환
- 에러 발생 시 None 반환

임베딩 함수

텍스트를 벡터로 변환하는 OpenAI 임베딩

embed_text() 함수

```
def embed_text(text: str) ->
    list: try:
        response =
            client.embeddings.create(
            input=[text],
        )   model="text-embedding-3-small"
        return
    response.data[0].embedding except
    Exception as e:
        print(f"임베딩 실패: {text}")
        print("에러:\n",
        e) return []
```

임베딩이란?

- 텍스트를 1536차원의 숫자 벡터로 변환
- 의미론적으로 유사한 텍스트는 벡터 공간에서도 가까이 위치
- Vector(List[float]) 형식으로 반환
- 예외 발생 시 오류 메시지와 빈 리스트([]) 반환

text-embedding-3-small 모델

OpenAI의 효율적인 임베딩 모델로, 작은 크기에 비해 우수한 성능을 제공합니다.

사용자 입력 포맷 변환 함수

질의 형식 표준화로 임베딩 유사도 비교 정확도 향상

포맷 변환 함수 구현

```
def format_user_query(user_input: str) ->
    str: try:
        response =
            client.chat.completions.create(
            model="gpt-4",
            messages=[
                {"role": "system", "content": "사용자 질문을 다음 형식으로 바꿔줘: {학과명}에서 작성한 공지: {주제} ,가능한 한 간결하고 명확하
                게 작성해."},
                {"role": "user", "content": user_input}
            ]
        )
        return
    response.choices[0].message.content.strip() except
    Exception as e:
        print("GPT 변환 실패, 원본 사용:", e)
        return f"학과에서 작성한 공지:
        {user_input}"
```

사용자 입력 포맷 변환 함수

질의 형식 표준화로 임베딩 유사도 비교 정확도 향상

변환 예시

사용자 원본 질의

"컴공과에서 프로그래밍 대회 열리나요?"

변환된 질의 포맷

"컴퓨터공학과에서 작성한 공지: 프로그래밍 대회"

표준 포맷 변환의 장점

정확도 향상

사전에 공지사항 데이터도 동일한 포맷으로 처리했기 때문에, 벡터 간 유사도 계산 시 일관된 형식 으로 비교가 가능합니다.

예외 처리

GPT 변환에 실패하더라도 기본 포맷을 적용하여 검색 프로세스가 중단되지 않도록 설계되었습니 다.

코사인 유사도 계산

두 벡터 간의 유사성을 측정하는 방법

코사인 유사도 함수 구현

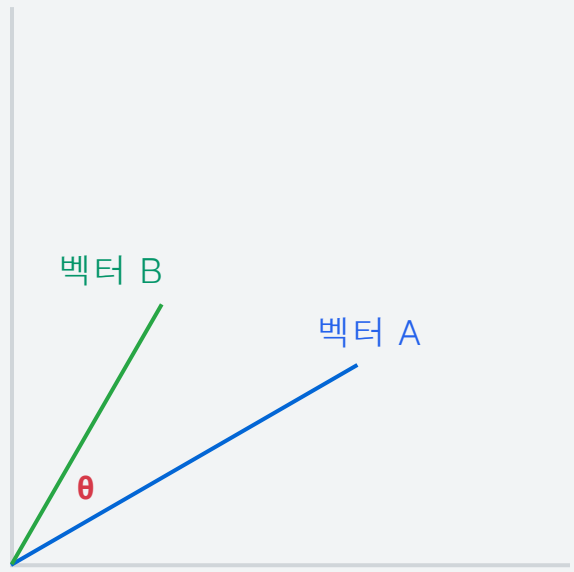
```
def cosine_similarity(vec1,
    vec2): v1 = np.array(vec1)
    v2 = np.array(vec2)

    return np.dot(v1, v2) / (np.linalg.norm(v1) *
    np.linalg.norm(v2)
```

활용 방식

- 사용자 질의 벡터와 DB에 저장된 공지사항 벡터 간 유사도 계산
- 결과값의 범위: −1 ~ 1 (1에 가까울수록 유사도 높음)
- 임계값(0.65) 이상인 공지만 선별하여 관련성 높은 결과 제공

코사인 유사도 개념

코사인 유사도 = 두 벡터의 내적 ÷ (벡터A의 크기 × 벡터B의 크기)



두 벡터 간 각도(θ)의 코사인 값이 유사도를 나타냅니다.
각도가 작을수록(방향이 유사할수록) 유사도는 1에 가까워집니다.

유사 공지 검색

임베딩과 코사인 유사도를 활용한 관련 공지 검색 함수

사용자 입력 포맷 변환 → 벡터 임베딩 생성 → DB 공지 벡터 검색 → 유사도 정렬 및 반환

search_similar_notices 함수

```
def search_similar_notices(user_input,
    top_n=5): # 사용자 입력을 포맷팅하여 임베딩
    formatted_input =
    format_user_query(user_input) query_vector =
    embed_text(formatted_input)
    # DB에서 공지사항 벡터
    가져오기 conn =
    get_connection()
    cursor = conn.cursor(dictionary=True)
    cursor.execute("SELECT id, title,
    department,
                   link, posted_date,
                   vector FROM notice
                   WHERE vector IS NOT NULL")
    rows =
    cursor.fetchall()
    cursor.close()
    conn.close()
```

유사도 계산 및 정렬

```
scored = []
for row in rows:
    try:
        notice_vector = json.loads(row["vector"])
        score = cosine_similarity(query_vector, notice_vector)
        if score > 0.65:
            row["similarity"] = score
            scored.append(row)
    except Exception as e:
        print(f" 벡터 파싱 오류 (id={row['id']}):", e)

scored.sort(key=lambda x: x["similarity"], reverse=True)

cleaned = [
    {
        "title": row["title"],
        "department": row["department"],
        "link": row["link"],
        "posted_date": row["posted_date"],
    }
    for row in scored[:top_n]
]
return cleaned
```

유사 공지 검색

임베딩과 코사인 유사도를 활용한 관련 공지 검색 함수

유사도 필터링

코사인 유사도가 0.65 이상인 공지만 선별하여 관련성 높은 결과만 제공합니다.

유사도 기반 정렬

계산된 유사도를 기준으로 내림차순 정렬하여 가장 관련성 높은 공지를 먼저 보여줍니 다.

결과 형식

반환되는 결과는 제목(title), 학과(department), 링크(link), 게시일(posted_date)을 포함한 형태로 가공됩니다.

실행부 & 서버 연결(Flask)

사용자 질의 처리와 API 서버 구축

실행부

```
if __name__ == "__main__":
    user_question = input("질문을 입력하세요: ")
    results = search_similar_notices(user_question)

    if not results:
        print("관련된 공지가 없습니다.")
    else:
        print("\n유사 공지 결과:")
        for r in results:
            print(f"-{r['posted_date']} |
{r['department']} | {r['title']}")
            print(f"링크: {r['link']}")
```

사용자의 질문을 받아 search_similar_notices() 함수로 유사 공지를 검색하고 결과를 출 력합니다.

웹/앱 프론트엔드 → /search API 요청 → 검색 결과 JSON 응답

Flask 서버 연결

```
from flask import Flask, request, jsonify
from notice_search import
search_similar_notices
app = Flask(__name__)

@app.route("/search",
methods=["POST"]) def search():
    data = request.get_json()

    if not data or "user_input" not in data:
        return jsonify({"error": "user_input가 필요합니다."}),
        400
    user_input = data["user_input"]
    results =
    search_similar_notices(user_input)
    return
    jsonify(results)
if __name__ == "__main__":
    app.run(host="0.0.0.0",
    port=5000)
```

Flask를 이용해 /search 엔드포인트를 구현하여 웹 요청으로 공지 검색 기능을 제공합니다.

마무리 정리

공지사항 검색 시스템의 전체 흐름 및 구조

시스템 흐름도

프론트엔드
사용자 질의 전송

→

Flask 서버
요청 처리

→

임베딩 변환
텍스트 → 벡터

→

DB 검색
유사도 계산

→

JSON 응답
결과 반환

핵심 구성요소

- 환경 변수 관리: API 키와 DB 접속 정보를 안전하게 관리
- 텍스트 임베딩: OpenAI의 text-embedding 모델로 벡터 변환
- 코사인 유사도: 벡터 간 유사성 계산으로 연관 공지 발견
- 데이터 저장소: MySQL에 공지 및 임베딩 벡터 저장
- 포맷 변환: 질의 형식을 일관되게 변환하여 매칭 정확도 향상
- API 연동: Flask 서버로 외부 시스템과 연결성 확보

결론

OpenAI 임베딩 API와 코사인 유사도 알고리즘을 활용하여 사용자 질의에 가장 적합한 공지를 찾는 시스템을 구현했습니다. 이 시스템은 효율적인 검색 기능을 제공하며, 프론트엔 드와 API 연동을 통해 완성된 서비스로 확장 가능합니다.